DATA TRANSFORMATION

Definition : Cleaning and filtering the data before storing it in Power BI's internal model, so it can be used for modeling and further steps.

Tool Used: Power Query Editor.

What is a Power Query?

- Power Query is Power BI's data preparation engine.Performs ETL Process
- **Purpose**: Connect to a data source, reshape the data, and load it into Power BI's internal model — **without touching the original source**.
- **M Language**: Every transformation is recorded as a **step** in a script called **M** (the underlying language). Power Query generates M automatically as you click.

How to open Power Query Editor

There are two ways:

1. From the ribbon: Home → Transform Data
2. During data import: in the Navigator window, click Transform Data instead of Load

Step-by-Step: Loading Your First Dataset

1. **Open Power BI Desktop** (Download free from microsoft.com/powerbi).
2. **Get Data**: Click Home → Get Data → Text/CSV (Starting with CSV, the simplest format).
3. **Load sample data**: Use this free dataset: Superstore Sales CSV. (Classic retail data: orders, products, regions, profit).
4. **Enter Power Query Editor**: After selecting the file, click **Transform Data** (not Load). This opens the Power Query Editor window.

1.Understanding Power Query Editor Interface

The three panels inside Power Query Editor

Panel	Location	Purpose
Queries pane	Left	Lists all queries/tables loaded into Power BI. Click any query to preview it.
Data Preview	Center	Shows the actual rows and columns of the selected query. This is where you interact with columns.
Query Settings	Right	Contains the query name and the Applied Steps list — a recorded history of every transformation applied, in order.

2. Power Query Ribbon Tabs

The Power Query ribbon has four main tabs used in daily work:

Tab	Key actions available
Home	Close & Apply, Remove Rows, Keep Rows, Use First Row as Headers, Merge, Append, data source settings
Transform	Change Type, Replace Values, Split Column, Group By, Transpose, Pivot/Unpivot, Format
Add Column	Custom Column, Conditional Column, Column From Examples, Index Column, Duplicate Column
View	Advanced Editor (M code), Column Quality, Column Distribution, Column Profile

3. Data Types-The Foundation of Clean Data

Every column in Power Query has a data type shown as an icon next to the column name. Getting data types right is non-negotiable — wrong types cause broken visuals, broken DAX formulas, and incorrect date hierarchies.

Icon	Type	Use for	Common mistake
123	Whole Number	Quantity, Count, IDs	Storing decimal values — loses precision
1.2	Decimal Number	Sales, Profit, Discount	Using for IDs — unnecessary decimals

ABC	Text	Names, IDs, Categories	Dates left as text — breaks everything
📅	Date	Order Date, Ship Date	Not specifying locale for non-US formats
True/False	Boolean	Flags, Yes/No columns	Storing as text instead of boolean

4. The Three Basic Transformations

4.1. Choose the Columns

Purpose: Select which columns to keep based on requirements.

Method 1: Choose Columns (Keep/Deselect)

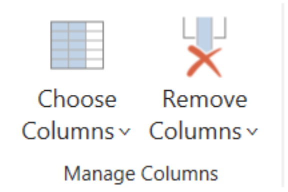
Step 1: Power Query Editor → Home →

Step 2: Manage columns → Choose Columns

Step 3: Check the box next to columns to keep (or deselect unwanted ones)

Step 4: Click OK

Example: In Superstore, removed Row ID from the *Orders* table.



Method 2: Remove Columns (Remove Selected/Other)

Step 1: Select the columns to remove (Ctrl+select for multiple)

Step 2: Choose Remove Columns (removes selected ones) OR Remove other Columns (removes all *except* selected ones)

Example: Removed Postal Code (using Remove columns → Remove columns).

4.2. Rename the Columns

Purpose: Rename columns that have large names or are hard to process/understand.

Method 1: Right-Click

Step 1: Select the column

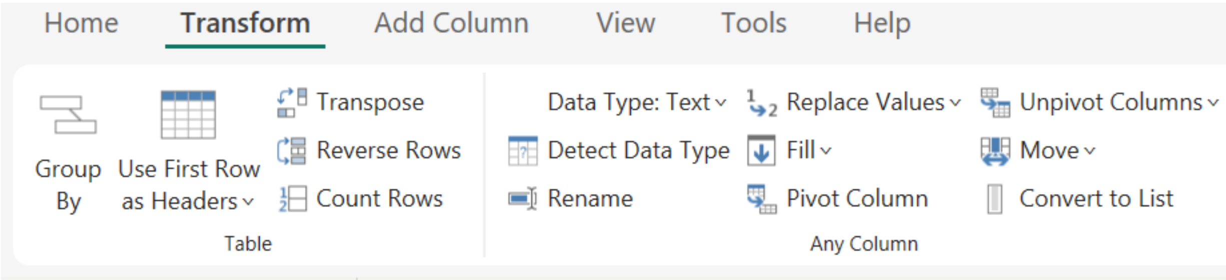
Step 2: Right-click on the column header → Rename

Step 3: Type the new name then press ENTER

Method 2: Transform Tab

Step 1: Select column

Step 2: Transform → Any column: Rename



4.3. Change the type (Data Type)

Purpose: Power Query often defaults all columns to Text, which can be dangerous for insights. Need to change the data type.

Method 1: Right-Click

Step 1: Select column → Right click

Step 2: Change Type → Choose the data type of the column → press ENTER

Method 2: All columns at once

Step 1: Select All (Ctrl+A)

Step 2: Transform → Any columns → Detect DataType

Example: In the *Peoples Table*, the data type of both columns was "Any" and was changed to Text.

5.SOME OTHER TRANSFORMATIONS

5.1.Column Header

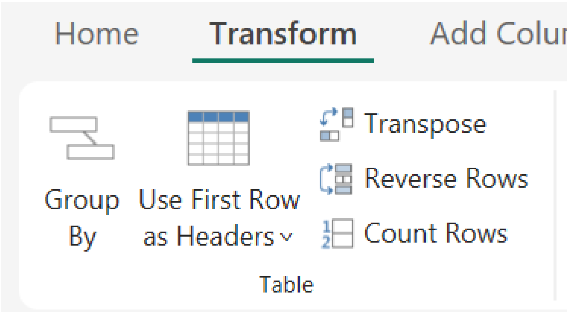
Purpose: The table must have proper column headers (especially if headers are in the first row) to get proper insights.

Step 1: Select the columns

Step 2: Transform → Use First row as Headers

(Options are: Use First Row as Header OR Use Header as First row)

Step 3: Select Use First Row as Header



5.2.Splitting the column

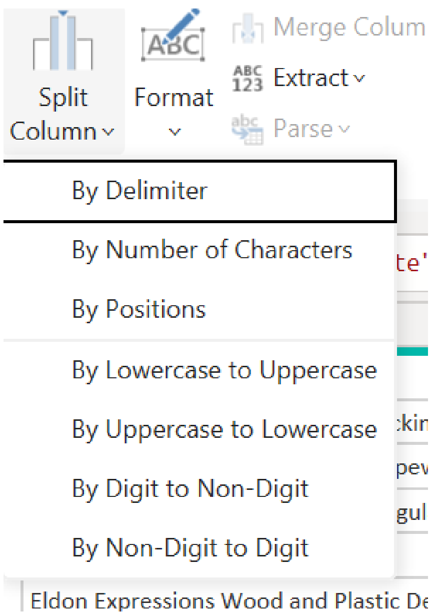
Purpose: Separate data within a single column that needs to be isolated for searching, filtering, or deeper analysis.

Step 1: Select the column

Step 2: Home → Transform → Split Column

Step 3: Select the preferred split method → click OK

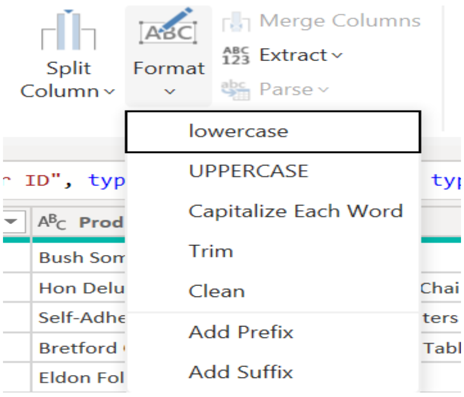
Example: Split Customer Name into two distinct columns using the By Delimiter option.



5.3. Remove the white spaces in the start and end:

Step 1: Select the column

Step 2: Transform → Format → Trim



5.4.Removing top junk rows

Purpose: raw data has title rows, report headers, or blank rows above the actual column headers.

Step 1. Home → Remove Rows → Remove Top Rows

Step 2. Enter the number of rows to remove (e.g. 3)

Step 3. Click OK

Note : Order matters

Always remove top rows before promoting headers. If you promote first, your junk data becomes the column names and must be manually fixed.

5.5.Removing blank columns

Purpose: exported data contains empty columns that carry no information and add noise to the model.

Step 1: Click the blank column header to select it

Step 2: Home → Remove Columns (or right-click → Remove)

5.6. Removing duplicate rows

Purpose: a column that should be unique (like Order ID in the Returns table) contains repeated values, which prevents building a 1:Many relationship.

Step 1: Select the column(s) that define uniqueness

Step 2: Home → Remove Rows → Remove Duplicates

5.7.Removing null values

Purpose: rows with blank/null values in key columns would corrupt relationships or produce incorrect aggregations.

Applied to: People table (Region and Person columns). A null row in a 4-row dimension table would create a phantom 'blank' region in every regional analysis.

Method 1:

Step 1: Click the dropdown arrow on the column header

Step 2: Uncheck (null) from the filter list

Method 2:

Home → Remove Rows → Remove Blank Rows (removes rows where all cells are blank)

6. Merge vs Append Queries

Feature	Append	Merge
Direction	Vertical — stacks rows	Horizontal — joins columns
SQL equivalent	UNION ALL	JOIN (Left Outer, Inner, etc.)
Requires	Similar structure (column names guide matching — mismatched columns get nulls)	At least one matching key column in both tables
Example	Combine Sales_2023 + Sales_2024 into one table	Bring Product Category into Sales table using Product ID

6.1 How to Append queries

Step 1: Open the first query

Step 2: Home → Append Queries → Append Queries as New (creates a separate combined query)

Step 3: Select the second table from the dropdown

Step 4: Click OK — rows are stacked. Column matching is by name, not position.

Note: Append does not require identical column counts

If Orders_2023 has 10 columns and Orders_2024 has 11 columns, Append still works. The extra column appears in the combined table and rows from the table that lacked it show null for that column.

6.2 How to Merge queries

Step 1: Open the primary query (e.g. Orders)

Step 2: Home → Merge Queries → Merge Queries
Step 3: Select the second table (e.g. Products) from the dropdown
Step 4: Click the matching column in both tables (e.g. Product ID)
Step 5: Choose join type — Left Outer is most common (keeps all rows from Orders, adds matching data from Products)
Step 6: Click OK — a new column appears containing the related table
Step 7: Click the expand icon on that column header → select which columns to bring in

Join type	Behaviour
Left Outer	All rows from left table, matching rows from right. Unmatched right rows → null. Most common.
Inner	Only rows that match in both tables. Unmatched rows from either side are dropped.
Right Outer	All rows from right table, matching rows from left.
Full Outer	All rows from both tables. Nulls where no match exists on either side.
Left Anti	Rows in left table that have NO match in right table.

7. Adding Custom Columns

Custom columns are calculated inside Power Query using M language. Unlike DAX calculated columns (which are computed at query time in the model), Power Query custom columns are computed during data load — they become part of the table permanently.

7.1 Custom Column

Purpose: To create columns that can give required information

Step 1: Go to the Orders query
Step 2: Add Column → Custom Column
Step 3: Then write the formula
Example
Name: Delivery Days
Formula:
Duration.Days([Ship Date] - [Order Date])

Formula breakdown:

- [Ship Date] - [Order Date] → subtracting two Date columns produces a Duration value (e.g. '3 days, 00:00:00')
- Duration.Days(...) → extracts the integer day count from the Duration as a plain whole number
- Result: a clean number column (e.g. 3, 5, 14) representing days to ship

Note : Why not use DAX for this?

Delivery Days is a row-level attribute — a fixed property of each order that does not change based on filters. This makes it a better fit for a Power Query custom column than a DAX calculated column. Power Query columns also load faster and use less memory.

7.2 Conditional columns

Purpose: A conditional column assigns values based on rules — like an IF statement.

Steps :Use Add Column → Conditional Column for a no-code approach, or write M directly:

```
if [Sales] → 1000 then "High Value" else if [Sales] → 500 then "Medium" else "Low"
```

M Language-the Advanced Editor

Every transformation you apply via the ribbon generates M code automatically. You can view and edit this code in the Advanced Editor (View → Advanced Editor). Understanding M is essential for debugging and for transformations the ribbon cannot do.

The let...in structure

Every M query follows this pattern:

```
let
    Step1 = <expression>,
    Step2 = <expression using Step2>,
    Step3 = <expression using Step3>
in
    Step3
```

- let → defines a series of named transformation steps
- Each step references the previous step by name — this is what creates the pipeline
- in → declares which step's output is returned as the final query result
- Deleting or reordering steps breaks references — always check dependencies first

Real M code from Superstore — Understanding

Source step — establishes the file connection:

```
Source = Excel.Workbook(File.Contents("C:\Superstore.xlsx"), null, true)
```

- Excel.Workbook() → M function to read an Excel file
- File.Contents('path') → reads the binary content of the file
- null → no specific sheet targeted at this step; navigation happens in the next step
- true → promotes the first row as column headers during workbook read

Changed Type step — applies data types to all columns:

```
#"Changed Type" = Table.TransformColumnTypes(#"Removed Columns",{
        {"Order ID", type text},
        {"Order Date", type date},
        {"Ship Date", type date},
        {"Sales", type number},
        {"Quantity", Int64.Type},
        {"Discount", type number},
        {"Profit", type number}
})
```

- Table.TransformColumnTypes() → applies a list of column-to-type mappings
- #"Removed Columns" → references the previous step by name as input
- {"Order ID", type text} → column name first, target data type second
- Int64.Type → whole number specifically (64-bit integer); more precise than "type number"

8. Query Folding

Query folding is the ability of Power Query to translate transformations into the native query language of the data source (typically SQL) and execute them at the source rather than in Power BI's memory engine.

Note: Why it matters?
Filtering 50 million rows at SQL Server and returning 10,000 rows to Power BI is dramatically faster than importing 50 million rows and filtering in memory. Query folding is a key performance topic in senior-level Power BI interviews.

Transformations that fold (push to source)	Transformations that do NOT fold
Filter rows	Custom columns using M functions
Remove columns	Merging queries from different source types
Rename columns	Table.Group with complex aggregations
Change data types (SQL sources)	Any step after a non-folding step

Sort rows	

How to check if a step is folding: right-click any Applied Step in Power Query. If 'View Native Query' is available and not greyed out, that step is folding to the source.

Note: Folding breaks permanently

Once a non-folding step appears in your Applied Steps, all subsequent steps also lose folding — this is called 'folding breaking'. Design your pipeline so filtering and column removal happen before any custom M transformations.

9. Mistakes Made & Lessons Learned

Mistake	What happened	Correct approach
Changed date type without specifying locale	DD-MM-YYYY dates were misread as MM-DD-YYYY	Change Type → Using Locale → English (United Kingdom)
Promoted headers before removing top rows	Junk row data became column names	Remove top rows first, then promote headers
Left null values in People table	Blank region appeared in all regional visuals	Remove null rows from dimension tables during Power Query

Power Query Quick Reference Card

Task	Menu path
Open Power Query	Home → Transform Data
Promote first row as headers	Home → Use First Row as Headers
Remove top N rows	Home → Remove Rows → Remove Top Rows
Remove duplicates	Home → Remove Rows → Remove Duplicates
Remove blank rows	Home → Remove Rows → Remove Blank Rows
Remove a column	Right-click column → Remove
Change data type	Click type icon on column → select type
Change type with locale	Right-click → Change Type → Using Locale
Add custom column	Add Column → Custom Column

Add conditional column	Add Column → Conditional Column
Create dimension viaReference	Right-click query in Queries pane → Reference
Append two tables (stack rows)	Home → Append Queries → Append Queries as New
Merge two tables (join columns)	Home → Merge Queries → Merge Queries
View M code	View → Advanced Editor
Check query folding	Right-click Applied Step → View Native Query
Apply all changes to model	Home → Close & Apply